

Security Audit

Zero Edge (GameFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	20
Conclusion	21
Our Methodology	22
Disclaimers	24
About Hashlock	25

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Zero Edge team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Zero Edge is a decentralized Web3 gaming platform designed to offer provably fair, transparent, and trustless betting experiences. Built on blockchain technology, it eliminates the traditional house edge by providing true 50/50 odds, ensuring that players have a fair chance in every game. The platform caters to both crypto-native gamblers and traditional players seeking a more transparent alternative to centralized betting systems. By leveraging smart contracts and on-chain verifiability, Zero Edge aims to redefine the online gambling landscape, aligning with the growing demand for fairness and decentralization in gaming.

Project Name: Zero Edge

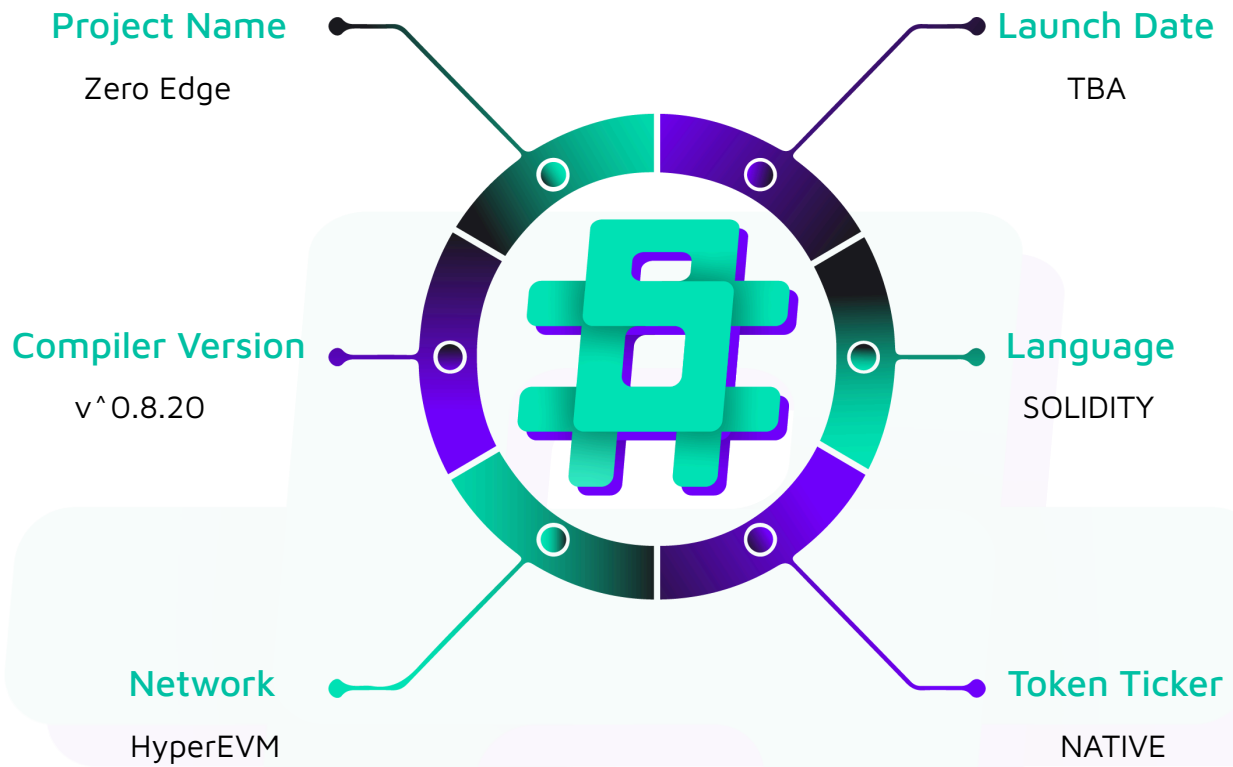
Project Type: GameFi

Compiler Version: ^0.8.20

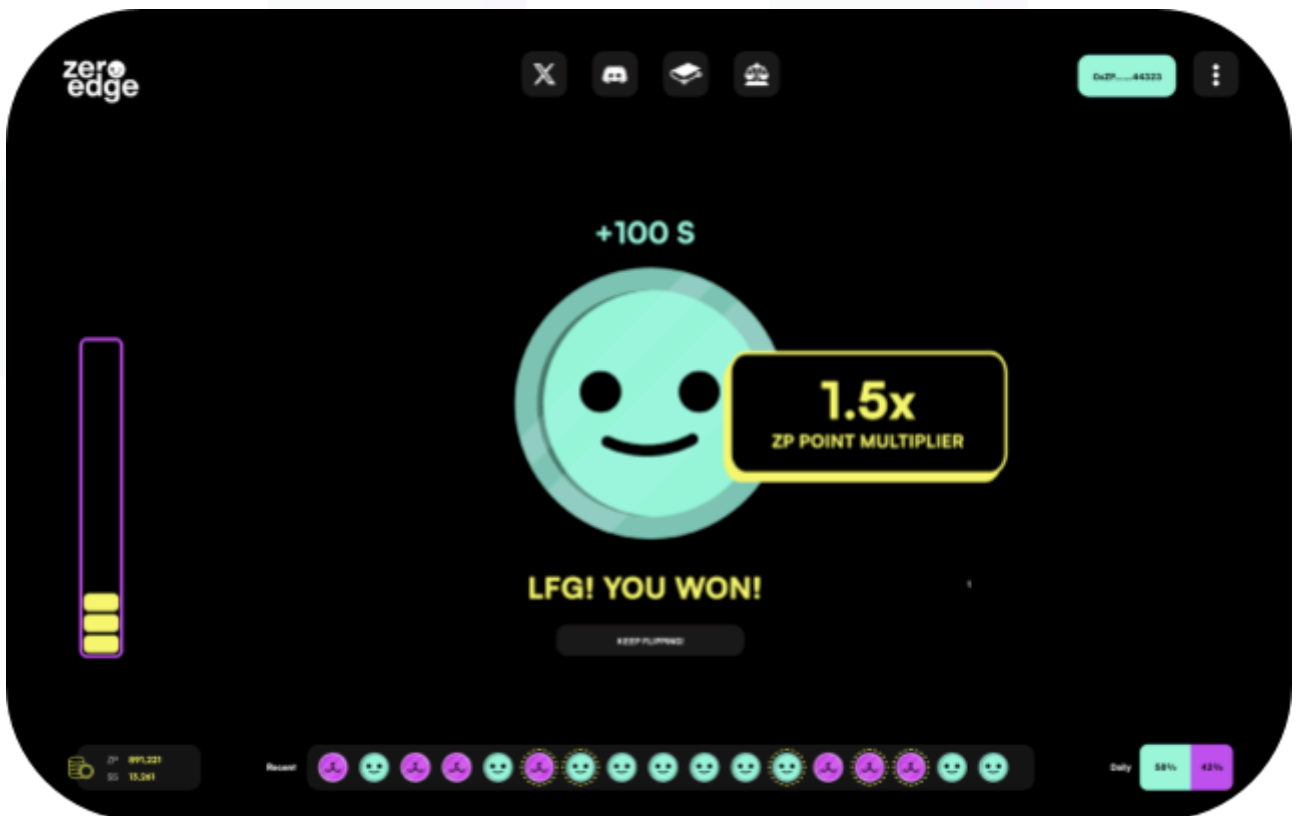
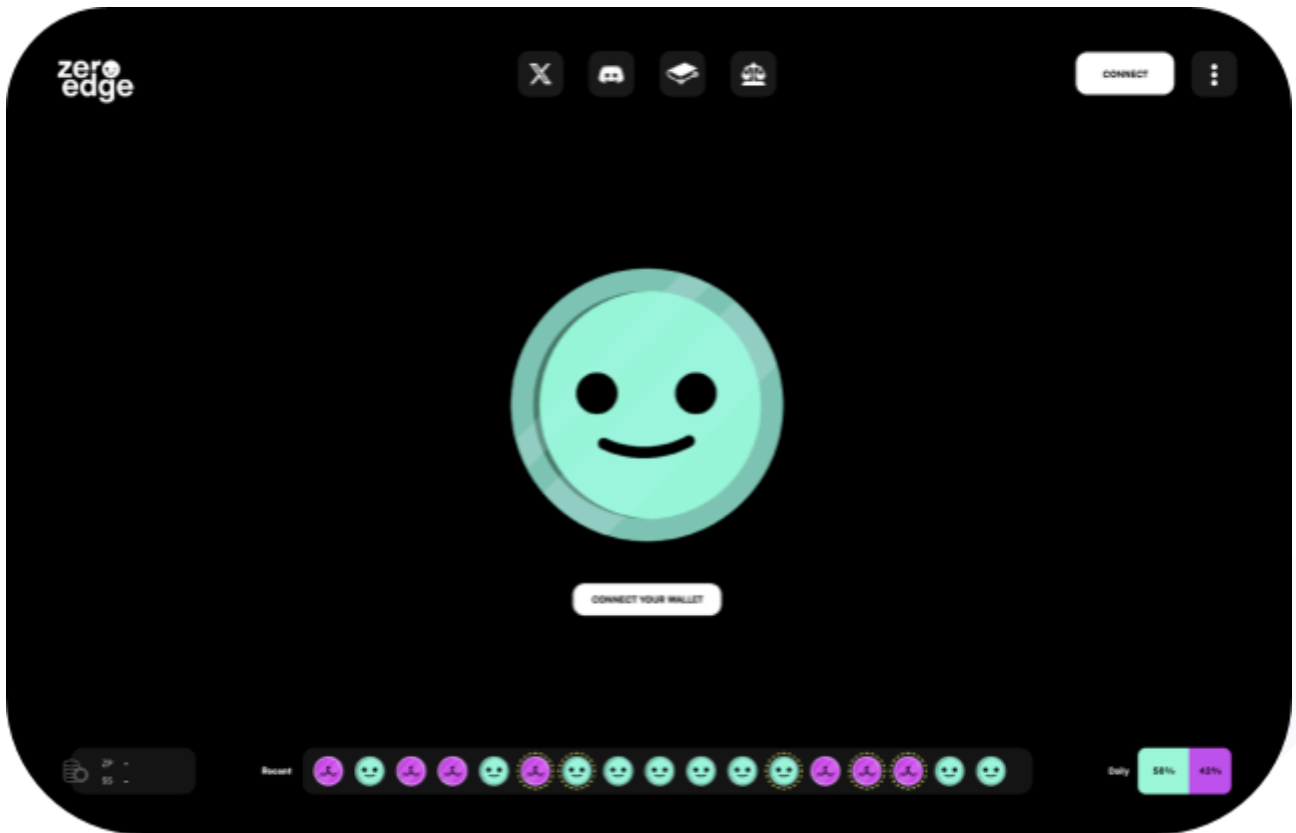
Website: <https://www.zeroedge.fun/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Zero Edge project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Zero Edge Smart Contracts
Platform	HyperEVM / Solidity
Audit Date	May, 2025
Contract 1	CoinFlipGame.sol
Contract 1 MD5 Hash	6175096f63f695338a87f33cbcf9068c
Contract 2	Treasury.sol
Contract 2 MD5 Hash	1d0a34089ff028bf90eef7c5ea9dbe3b
Contract 3	AllowlistHook.sol
Contract 3 MD5 Hash	9b93294ba02e2f42b418e24172d5c638
Audited GitHub Commit Hash	2a59751081105ef82d658c1df1c0407e491f1ca7
Fix Review GitHub Commit Hash	e9fb2d0e44d83937304f6de995fb272e90dc9dd2

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Medium severity vulnerabilities

1 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
CoinFlipGame.sol - Allows users to: - Place coin flip bets (HEADS/TAILS) with Native Token - Cancel pending bets after 5-minute delay if VRF fails - Receive automatic 2x payouts when winning - Allows admins to: - Pause/unpause the game - Update Treasury and VRF operator addresses - Set pre-bet hook contracts - Upgrade contract implementation - Allows emergency roles to: - Pause game operations	Contract achieves this functionality.
Treasury.sol - Allows operators to: - Deposit funds to bankroll - Request timelock-protected withdrawals of fees/bankroll - Execute withdrawals after timelock period - Convert trapped Native token to bankroll - Allows admins to: - Expedite emergency withdrawals bypassing timelock - Set fee rates (up to 10%) and bet limits - Update max bet thresholds based on bankroll - Upgrade contract implementation	Contract achieves this functionality.

- Allows emergency roles to: - Pause all payouts during emergencies	
AllowlistHook.sol - Allows users to: - Place bets only with valid EIP-712 signatures - Allows owner to: - Set authorized signer address for EIP-712 signatures Key Features: - Implements EIP-712 signature verification - Prevents replay attacks using nonces - Restricts betting to authorized players only	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Zero Edge project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Zero Edge project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] CoinFlipGame - Incorrect Import Path in 'CoinFlipGame.sol' Will Cause the Code to Never Compile

Description

The project uses an incorrect import path for the Gelato VRF contracts, referencing `@gelatonetwork/vrf-contracts` instead of the official `@gelatodigital/vrf-contracts` package. This misconfiguration creates a supply chain security risk that could compromise the coin flip game's integrity by compromising the randomness generation mechanism.

It also creates a problem for auditors and new developers. If you install this package, you will have problems because it does not exist. Because of this, there is a risk that the contract will not compile

Vulnerability Details

This problem is present in both the `CoinFlipGame.sol` files and the `foundry.toml` file.

```
//...imports
import "@gelatonetwork/vrf-contracts/contracts/GelatoVRFCustomerBase.sol";
//...imports

/**
 * @title CoinFlipGame
 */
contract CoinFlipGame is
```

```
remappings = [
    "forge-std/=lib/forge-std/src/",
    "@openzeppelin/contracts/=lib/openzeppelin-contracts/contracts/",
```

```
"@openzeppelin/contracts-upgradeable/=lib/openzeppelin-contracts-upgradeable/contracts/",  
  "@gelatonetwork/vrf-contracts/=lib/vrf-contracts/"  
]
```

The official [Gelato documentation](#) specifies the correct package for installation.

Impact

- VRF operator functionality may be compromised
- The contract will not compile

Recommendation

Ensure you use the correct package import and amend the remapping in the `foundry.toml` file.

Status

Resolved

Low

[L-01] Treasury - Data from `maxBetThresholds` array is not deleted after length change

Description

The `setMaxBetThresholds` function changes the `maxBetThresholds` array by changing the length in the slot. The Slot of the array stores its length, and all variables are stored in order in the slot, starting from the hashed value of the array length. In this case, if an array is changed from larger to smaller, the previous values are stored in the slots.

```
function setMaxBetThresholds(ITreasuryTypes.BetThreshold[] calldata thresholds) external
onlyRole(CONFIG_ROLE) {
    //...code
    // Use assembly to efficiently resize the array
    assembly {
        sstore(maxBetThresholds.slot, thresholds.length)
    }

    // Assign elements directly
    for (uint256 i = 0; i < thresholds.length; i++) {
        maxBetThresholds[i] = thresholds[i];
    }

    emit MaxBetThresholdsUpdated(thresholds);
}
```

Proof of Concept

```
function test_SlotArray() public {
    uint256 minBetAmount = 1 ether;
    uint256 minimalBankroll = 3 ether;

    vm.startPrank(admin);
    treasury.setMinBet(minBetAmount);
}
```

```

ITreasuryTypes.BetThreshold[] memory thresholds = new ITreasuryTypes.BetThreshold[](2);
thresholds[0] = ITreasuryTypes.BetThreshold({
    bankrollThreshold: 0,
    maxBetAmount: 10 ether
});
thresholds[1] = ITreasuryTypes.BetThreshold({
    bankrollThreshold: 2 ether,
    maxBetAmount: 20 ether
});
treasury.setMaxBetThresholds(thresholds);

ITreasuryTypes.BetThreshold[] memory thresholdsNew = new
ITreasuryTypes.BetThreshold[](1);
thresholdsNew[0] = ITreasuryTypes.BetThreshold({
    bankrollThreshold: 0,
    maxBetAmount: 10 ether
});

treasury.setMaxBetThresholds(thresholdsNew);
uint256 len = treasury.getMaxBetThresholds().length;

assertEq(len, 1);
Uint256 value = uint256(vm.load(address(treasury),
bytes32(uint256(keccak256(abi.encode(uint256(6)))) + 2)));

assertEq(value, 2 ether);

vm.stopPrank();
}

```

```
forge test --mt test_SlotArray -vv
```

```
Ran 1 test for test/Treasury.t.sol:TreasuryTest
```

```
[PASS] test_SlotArray() (gas: 57134)
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.99ms (171.67µs CPU time)
```

```
Ran 1 test suite in 230.71ms (2.99ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

This contract is updatable and all variables are contained in the proxy contract, if the logical contract is changed, it may cause unexpected behavior.

Recommendation

Consider deleting data from the array

Status

Resolved

QA

[Q-01] CoinFlipGame - Remove console import

Description

In contracts `CoinFlipGame.sol` the `forge-std/console.sol` library is imported and never used.

Recommendation

Remove all occurrences of `forge-std/console.sol` imports from production code

Status

Resolved

[Q-02] Contracts - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider [known bugs](#) for the compiler version that is chosen.

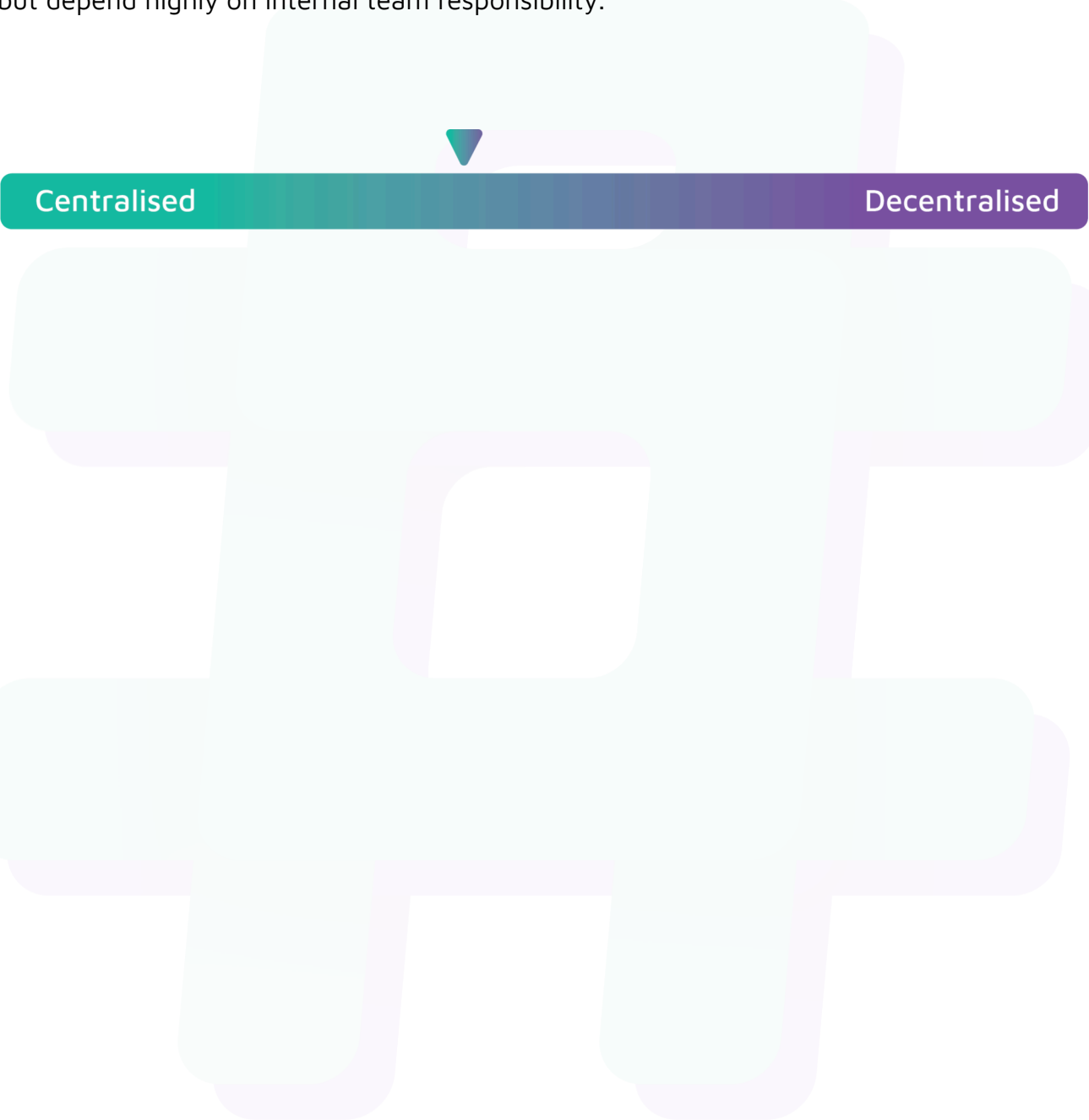
Status

Resolved

Centralisation

The Zero Edge project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Zero Edge project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.